MODULE

23

SQLAlchemy 2.0 & Alembic

Use an ORM like a pro: models, queries, the N+1 trap, indexes, transactions and migrations.

YOU WILL LEARN

Models	`models.py`
ORM CRUD	`01_orm_crud.py`
N+1 & loading	`02_n_plus_1_and_loading.py`
SQLAlchemy + FastAPI	`03_sqlalchemy_with_fastapi.py`
Alembic migrations	`04_alembic_migrations.py`
Interview Q&A	`INTERVIEW.md`

WHY ORM

Why an ORM?

THE PROBLEM

Hand-written SQL strings everywhere: typos found only at runtime, copy-pasted joins, and every row comes back as a tuple you index by number.

THE IDEA

An ORM maps tables to Python classes. You query with Python, get objects back, and the ORM writes safe, parameterised SQL.

TRY IT MODELS AND CRUD

Models, a session, and a query

```python
from sqlalchemy import ForeignKey, create_engine, func, select
from sqlalchemy.orm import (DeclarativeBase, Mapped, Session,
                            mapped_column, relationship)

class Base(DeclarativeBase): pass

class Author(Base):
    __tablename__ = "authors"
    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(unique=True)
    books: Mapped[list["Book"]] = relationship(back_populates="author")

class Book(Base):
    __tablename__ = "books"
    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str]
    author_id: Mapped[int] = mapped_column(ForeignKey("authors.id"), index=True)
    author: Mapped[Author] = relationship(back_populates="books")

engine = create_engine("sqlite://")
Base.metadata.create_all(engine)
with Session(engine) as s:
    s.add(Author(name="Ramalho", books=[Book(title="Fluent Python")]))
    s.commit()
    stmt = select(Author.name, func.count(Book.id)).join(Book).group_by(Author.id)
    print(s.execute(stmt).all())
```

OUTPUT

```
[('Ramalho', 1)]
```

1. **Mapped[...]** type hints define the columns.
2. A **Session** is a unit of work: add, then commit.
3. **select()** builds SQL with joins and group by.

PICTURE IT N+1

The N+1 query problem

🐢 You load 50 authors with **1 query**, then touch **author.books** in a loop: the ORM quietly runs **1 more query per author**. 51 queries, a slow page, and it gets worse as data grows.



TRY IT N+1

Count the queries, then fix it

```python
from sqlalchemy import event, select
from sqlalchemy.orm import Session, selectinload

queries = 0
@event.listens_for(engine, "before_cursor_execute")
def count(*args): global queries; queries += 1

with Session(engine) as s:
    s.add_all([Author(name=f"author{i}", books=[Book(title="t")]) for i in range(50)])
    s.commit()

for options in ([], [selectinload(Author.books)]):
    queries = 0
    with Session(engine) as s:
        for a in s.scalars(select(Author).options(*options)):
            len(a.books)                    # touch the relationship
    print("eager" if options else "lazy ", "->", queries, "queries")
```

OUTPUT

```
lazy  -> 51 queries
eager -> 2 queries
```

1. Lazy loading: **1 + N** queries.
2. **selectinload**: 2 queries (one IN-list). **joinedload**: 1 JOIN.
3. Django: **select_related** / **prefetch_related**.

STEP BY STEP INDEXES

Indexes: why some queries are 1000× faster

	WITHOUT AN INDEX	WITH AN INDEX
How the DB finds rows	reads **every** row (full table scan)	walks a **B-tree** straight to them
Cost	O(n)	O(log n)
Add one on	—	foreign keys, columns in WHERE / JOIN / ORDER BY, unique fields
Price you pay	—	slower INSERT/UPDATE, extra disk
Check it	`EXPLAIN` shows "Seq Scan"	`EXPLAIN` shows "Index Scan"

> Composite index **(user_id, created_at)** helps "orders of a user, newest first"; the column order matters.

TRY IT INDEXES

See the query plan change

```
import sqlite3

db = sqlite3.connect(":memory:")
db.execute("CREATE TABLE orders (id INTEGER PRIMARY KEY, email TEXT, total REAL)")
db.executemany("INSERT INTO orders (email, total) VALUES (?, ?)",
               [(f"user{i}@mail.com", i) for i in range(50_000)])

q = "EXPLAIN QUERY PLAN SELECT * FROM orders WHERE email = 'user42@mail.com'"
print("before:", db.execute(q).fetchone()[-1])

db.execute("CREATE INDEX ix_orders_email ON orders (email)")
print("after :", db.execute(q).fetchone()[-1])
```

OUTPUT

```
before: SCAN orders
after : SEARCH orders USING INDEX
ix_orders_email (email=?)
```

1. **SCAN** = read every row.
2. **SEARCH ... USING INDEX** = jump to the row.
3. In SQLAlchemy: **mapped_column(index=True)**.

STEP BY STEP TRANSACTIONS

Isolation levels: what can go wrong concurrently

LEVEL	PREVENTS	STILL POSSIBLE
`READ UNCOMMITTED`	—	dirty reads (seeing uncommitted data)
`READ COMMITTED` `(Postgres default)`	dirty reads	non-repeatable reads, phantoms
`REPEATABLE READ`	+ non-repeatable reads	phantoms (in theory; not in Postgres)
`SERIALIZABLE`	everything	transactions may fail and need a retry

> Lost update fix: **SELECT … FOR UPDATE** (`.with_for_update()`), an atomic **UPDATE x = x - 1**, or a version column.

TRY IT SQLALCHEMY + FASTAPI

One session per request

```python
from typing import Annotated
from fastapi import Depends, FastAPI, HTTPException
from fastapi.testclient import TestClient
from sqlalchemy.exc import IntegrityError
from sqlalchemy.orm import Session

def get_db():
    with Session(engine) as session:      # opened per request
        yield session                     # closed afterwards

DB = Annotated[Session, Depends(get_db)]
app = FastAPI()

@app.post("/authors", status_code=201)
def create(name: str, db: DB):
    db.add(Author(name=name))
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(409, "author exists")
    return {"name": name}

c = TestClient(app)
print(c.post("/authors?name=vishal").status_code, c.post("/authors?name=vishal").status_code)
```

OUTPUT

```
201 409
```

1. A **yield** dependency = session per request, always closed.
2. Unique constraint violated → **409 Conflict**.
3. Tests override **get_db** with a test database.

PICTURE IT ALEMBIC

Migrations: version control for your schema

📝 Your code changes the model (a new column); production already has data. A **migration** is a small script that changes the live schema step by step, **without losing data**, and can be rolled back.

`edit models.py`
add Book.isbn

→

`alembic revision --autogenerate`
creates a migration file; review it!

→

`alembic upgrade head`
apply in CI/CD

→

`alembic downgrade -1`
undo if needed

IN REAL JOBS DATABASES

Database work in real jobs

Find slow queries

```
EXPLAIN ANALYZE SELECT ... ;   -- Postgres
echo=True / django-debug-toolbar
```

Safe deploys

```
add nullable column -> backfill -> make NOT NULL
(never rename a live column in one step)
```

Avoid lost updates

```
UPDATE stock SET qty = qty - 1
WHERE id = :id AND qty > 0
```

Connection pooling

```
create_engine(url, pool_size=10,
              pool_pre_ping=True)
```

INTERVIEW SAY THIS IN THE INTERVIEW

“What is the N+1 problem and how do you fix it?”

- You load N parent rows with 1 query, then accessing a relationship in a loop triggers **one extra query per row**.
- Symptoms: pages get slower as data grows; the query log shows the same SELECT repeated.
- Fix with **eager loading**: SQLAlchemy **selectinload** / **joinedload**; Django **select_related** (FK, JOIN) / **prefetch_related** (many, IN query).
- Catch it in tests by **counting queries**.

> **Follow-up they ask next:** ”What is an index and when would you NOT add one?” (a B-tree for fast lookups; skip it on tiny tables, low-selectivity columns, or write-heavy tables where it slows inserts).

INTERVIEW SAY THIS IN THE INTERVIEW

“How do you change a database schema in production safely?”

- Use **migrations** (Alembic / Django migrations) in version control, applied automatically in CI/CD.
- Make changes **backwards compatible**: the old and new code run side by side during a deploy.
- Expand → migrate → contract: add the new column, backfill, switch the code, then drop the old one later.
- Review autogenerated migrations; test **upgrade and downgrade**; back up first; watch for long table locks.

> **Follow-up they ask next:** ”Explain transaction isolation levels” and ”How do you prevent two users buying the last item?” (row lock with SELECT … FOR UPDATE, or an atomic conditional UPDATE).

RECAP REMEMBER THESE

Module 23 in 6 lines

1. Models with **Mapped[...]**; a **Session** per unit of work; **select()** to query.
2. N+1: lazy loading in a loop. Fix with **selectinload** / **joinedload**.
3. **Indexes** on foreign keys and filtered columns; check with **EXPLAIN**.
4. Transactions + isolation levels; prevent lost updates with locks or atomic updates.
5. FastAPI: a **yield** dependency gives one session per request.
6. **Alembic** migrations, reviewed, reversible, backwards compatible.

PRACTICE NOW

Run 02_n_plus_1_and_loading.py, then add an index and compare EXPLAIN before and after.